

ColMap: A memory-efficient occupancy grid mapping framework

Alex Fisher^{a,*}, Ricardo Cannizzaro^b, Madeleine Cochrane^b, Chatura Nagahawatte^b, Jennifer L. Palmer^b

^a RMIT University, GPO Box 2476V, Melbourne, VIC 3001, Australia

^b Defence Science and Technology, 506 Lorimer St, Port Melbourne VIC 3207, Australia

ARTICLE INFO

Article history:

Received 8 September 2020

Received in revised form 23 December 2020

Accepted 15 February 2021

Available online 20 April 2021

Keywords:

Mapping

Occupancy grid

Bayesian estimation

Robotics

ABSTRACT

In order to possess a significant degree of autonomy, a robot must be able to perceive its environment and store a representation of that environment for use in tasks such as localisation, navigation, collision avoidance, and higher decision making. It must do this subject to constraints on memory and processing power typical of the embedded computer systems commonly found on small robotic devices. These constraints are particularly important for flying robots (i.e. unmanned aerial vehicles), for which weight must be minimised. The challenge of storing a detailed map of a large area on a small embedded computer has led to the development of many algorithms that exploit the sparsity of typical maps to create a more memory-efficient representation. In this paper, we demonstrate that the verticality of both natural and man-made structures can be exploited to create a framework that can store occupancy grid maps efficiently, without causing additional computational burden. The new framework achieves an order-of-magnitude reduction in memory footprint relative to widely-used occupancy grid mapping software, while also achieving a slight speed-up in map insertion and access times. We also make available LIDAR scans taken from a hexacopter of an indoor flight arena that can be used to assist in evaluating future mapping and SLAM developments.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Having a map of the surrounding environment is critical to enabling robot autonomy. Tasks such as path-planning and collision avoidance cannot be performed without knowledge of the environment. Maps may also be needed for localisation, in areas where global navigation satellite systems may be unreliable, such as within highly built-up areas or indoors.

Sometimes the map may be known a priori, but in most cases it must be built during the mission, using data from on-board sensors. Sensors typically include optical range finders, RADAR, and SONAR. Such sensors are subject to numerous types of error, including noise, multi-pathing, and false/missed detections. For this reason, the mapping algorithm must take into account measurement uncertainty. Computing power on-board a robot is typically limited, particularly for small unmanned aircraft systems (UAS) for which weight and power consumption must be minimised. These small UAS are often employed in urban

areas and thus need a 3D map of their environment for navigation and obstacle avoidance; this map may also form part of the reconnaissance data produced by the mission. Memory constraints are particularly critical, as high-resolution 3D maps contain a significant amount of information. A mapping algorithm that stores information efficiently will enable the robot to explore a larger area.

Desirable features of a mapping algorithm include:

- Robustness with respect to sensor noise;
- The ability to incorporate measurements from different types of sensor into a single map;
- The ability to distinguish free space from unexplored space;
- Achieving compression of the map data by taking advantage of the sparseness of occupied regions;
- Growing the bounding box of the map during the mission without significant performance penalty;
- Being able to use the data efficiently for navigational and object-recognition purposes.

1.1. Previous work

There are three map representations typically found in the literature: point clouds, landmark-based maps, and occupancy grids. Point clouds store the “hits” returned by a ranging sensor

* Corresponding author.

E-mail addresses: alex.fisher@rmit.edu.au (A. Fisher), Ricardo.Cannizzaro@dst.defence.gov.au (R. Cannizzaro), Madeleine.Cochrane@dst.defence.gov.au (M. Cochrane), Chatura.Nagahawatte@dst.defence.gov.au (C. Nagahawatte), Jennifer.Palmer@dst.defence.gov.au (J.L. Palmer).

as a list of 3D points in space [1]. Advantages of this representation include the fact that the extent of the map need never be defined and that object recognition can be performed easily using methods such as RANSAC [2]. The serious disadvantages, however, include the fact that the size in memory increases continually and without bound, and also that the point cloud is unable to represent negative information (i.e., to explicitly model free space). Landmark-based maps have found use predominantly for simultaneous localisation and mapping (SLAM), in which a minimal number of points in space that are recognised with high certainty can be used to efficiently localise the robot. These, however, rely on recognition of the landmarks and are not particularly useful as an input to path-planning and navigation algorithms.

The third type of map representation, the occupancy grid (OG), has received the most attention lately for pure mapping problems. In its simplest form, the OG is a 2D or 3D Cartesian grid, with each cell being labelled with a binary variable as being either occupied or unoccupied. The biggest advantage of OGs is the ability to make them probabilistic and therefore account for sensor imperfections. This is usually done using the Bayesian framework, in which each cell stores a probability of occupancy, which is updated recursively using a formula based on Bayes' rule whenever the cell is detected as occupied or unoccupied through a sensor reading. The prior probability is typically initialised to 0.5 (i.e., unknown), the idea being that as sensor readings accumulate, this probability converges to either 0 (unoccupied) or 1 (occupied). This kind of Bayesian OG (BOG) was pioneered by Moravec and Elfes [3] and has been used in many mapping algorithms [4,5].

While the algorithm described in this paper is based on Bayesian OG, it is appropriate to mention other probabilistic OG methods. One is termed the "reflection map" [6]. It is equivalent to the BOG if certain restrictions are made in the latter. In a reflection map, a count of the number of sensor "hits" and "misses" is kept for each cell (a "miss" is defined as sample in which the sensor beam passes through a cell without a detection). The probability of occupancy can then be related to the number of hits and misses. While this is simple, it cannot account for the unique missed-detection/false-alarm rates of different sensors, nor is the occupancy probability available without first performing additional calculations. Another more advanced technique is known as the expectation maximisation (EM) algorithm [7]. It casts the probabilistic mapping problem as an optimisation problem and uses common optimisation (e.g., gradient-descent) algorithms to find the maximum likelihood map, given a complete list of sensor readings. This has two significant advantages over the BOG. The first is that the cell-independence assumption can be relaxed; the second is that only the forward sensor model need be known (i.e., $p(\text{sensor reading} \mid \text{map})$; the standard BOG needs the inverse sensor model, $p(\text{map} \mid \text{sensor reading})$, which is more difficult to obtain). The EM method, however, cannot be used to easily apply incremental updates to a map because it requires optimisation over the whole set of measurements. Thus it is inappropriate for real-time use.

Most recent developments in BOGs have concentrated on reducing memory requirements while maintaining the ability to efficiently update the map [5,8,9]. To understand memory consumption is important, consider a standard BOG for an area of 100 m x 100 m x 30 m, with 5-cm resolution. This would require 9.6 GB of memory, assuming probabilities are stored at single precision. One widely used BOG algorithm, Octomap [5] uses an octree structure, together with a pruning algorithm to address this problem. As measurements are obtained, the tree is expanded to full resolution; and as the probabilities converge, tree nodes whose children all possess the same probability are pruned, saving memory. The octree-based method also has several other

advantages, including a natural way of distinguishing unknown space from free space, the ability to query the tree at arbitrary depth to obtain maps of different resolutions, and a very efficient way to serialise and store the map for later use. The biggest disadvantages of the octree are that (a) the pruned cells must always be cubic and (b) the octree is often quite deep, which slows queries.

Another development has been the use of R-trees [8]. The algorithm, called RMAP, stores the occupied regions of the map as a series of axis-aligned rectangular cuboids within a tree structure to enable efficient access. In theory, this allows more efficient storage than Octomap, because the geometric shape of the leaf nodes is not forced to be cubic. The disadvantage of this method is that free and unknown space are not distinguished from each other. Also, to take full advantage of the axis-aligned cuboid representation, structures within the map must be axis-aligned. For a map being created during a mission in a realistic environment, this cannot usually be guaranteed.

Both Octomap and RMAP initially create the map at maximum resolution (i.e., with the smallest cell size), and then "fuse" cells with the same occupancy status. Einhorn et al. [9] have demonstrated that the opposite is possible, in an algorithm that initially uses larger grid cells and then splits them if the numbers of hits and misses for a particular cell become inconsistent with the expected distribution.

The tree-based methods mentioned above have one feature in common — they treat the three coordinate-axes identically. While this contributes to the elegance of such methods; in reality, the vertical direction is special. For example, man-made structures are typically aligned with the vertical axis, but in general are not aligned with the more arbitrary horizontal (x and y) axes. A map is also likely to be larger in the horizontal direction than in the vertical direction, especially for ground-based robots. This feature can be exploited to form more efficient maps, as is done by "2.5D" map representations. The most basic of these is the elevation map, which stores an elevation at each cell of a 2D grid [10,11]. The elevation map has also been extended to include probabilistic information, such as the variance of the elevation estimate, and the probability of occupancy by an innavigable obstacle [12]. This type of map is well-suited to ground robots, but is obviously not appropriate for modelling a complete 3D environment with overhanging obstacles like bridges and roofs. Airborne robots need this information to successfully conduct their mission. In an attempt to overcome this limitation, multi-level surface maps (MLS) have been used, in which a list of "surface patches" is created for each cell in a 2D horizontal grid [13,14]. This list is created from a point cloud of sensor returns, and thus does not incorporate negative information (i.e., sensor "misses").

In another 2.5D method, Dryanovski et al. [15] have created an effective mapping system, based on the reflection map, in which a list of vertical volumes is stored for each cell in a 2D grid. Both "hit" and "miss" volumes are stored separately, and they are fused under certain conditions. Disadvantages of this method, however, include a fixed-size grid that cannot be grown efficiently, the lack of the full Bayesian model, and some loss of accuracy due to the volume-fusion method.

More recently, signed distance fields (SDFs) have garnered attention for map representation in robots. A Euclidean SDF (ESDF) is a grid in which each cell stores the Euclidean distance to the nearest obstacle [16]. Obstacle surfaces are thus present where the field crosses zero. ESDFs are a convenient representation for collision-avoidance algorithms, but are not straightforward to generate. A naïve implementation typically first computes an OG, then computes the ESDF. Oleynikova et al. [17] presented a method of incrementally calculating ESDFs using truncated

SDFs, while Usenko et al. [18] presented the generation of local ESDFs from robot-centric OGs. Despite the emergence of these representations, OG representations remain important: (a) for use with existing path planning, collision avoidance, and SLAM algorithms; (b) as an intermediate representation for use in collision avoidance; and (c) as the most intuitive way of representing a map.

1.2. Motivation and contributions of this paper

When mapping large environments, memory usage is the most critical limitation of current algorithms. In this paper, we present a novel mapping algorithm that combines the advantages of the full BOG framework and the tree-based data storage method of Hornung et al. [5] and Khan et al. [8], with the memory saving of 2.5D approaches [13,15]. Our method uses an N^2 tree for the 2D horizontal grid, together with a vertical representation that can switch between a full OG and a run-length encoded list; this is explained further in Section 2. The full grid is used when a region is initialised. Then, as the occupancy probabilities converge, the representation is changed to more efficiently store the data, in a way similar to the tree-pruning method of Hornung et al. [5] or the “fusion” method of Khan et al. [8]. The run-length representation results in only a modest performance penalty.

The use of run-length encoded lists in the vertical direction can be thought of as breaking the map up into a series of stacked vertical columns, hence the name ColMap. Superficially, this may appear to be a subset of the functionality of RMAP, which decomposes the map into a minimal set of axis-aligned rectangular cuboids. Our method is quite different, however, in that it (a) explicitly models free space (such that unknown space is distinct from free space), (b) eliminates the complexities of fusing volumes in the horizontal directions, and (c) contains an efficient representation of commonly occurring probabilities resulting in a significantly reduced memory requirement. Although Khan et al. [8] correctly state that the known/unknown status of cells in the grid can be easily determined from the robot path and sensor field of view (and stored outside the map), we believe it advantageous to update and store information as part of the map itself. This simplifies path-planning and other uses of the mapping data. Our motivation for restricting the “fusion” of volumes to vertical columns is the fact that structures in a robot’s environment are much more likely to possess vertical correlation than they are to possess correlation in the arbitrary directions of the horizontal axes. For example, a planar wall will almost definitely be aligned with the vertical axis; however, it would be only by coincidence that it would be aligned with the x- or y-axes of a map. The exception to this is man-made horizontal planar surfaces such as floors and ceilings; however our approach is quite efficient at storing indoor environments, as will be demonstrated by an example.

Given that Octomap explicitly models free space, it is directly comparable to our method in terms of both memory usage and performance. As an example of the potential memory saving of the ColMap representation over Octomap, take the example of the interior of an empty room that consists of four walls, a floor, and a ceiling. To store these planar surfaces at high resolution, the octree must be expanded in *all three dimensions*, because the elements of the tree must be cubic. Tree nodes in the unoccupied centre of the room can remain collapsed to various levels. The ColMap representation in this example is simple: the interior part consists of lists of three stacked columns: occupied (floor), unoccupied (interior), occupied (roof). Meanwhile each wall consists of a single occupied column. This is illustrated in Fig. 1, which shows a 2D slice of such a map using both representations. Each box represents a cell for which an occupancy probability must be

stored — the leaves of the tree in the case of Octomap and elements in the list of columns in the case of ColMap. In this simple example, the octree representation requires 248 leaf nodes, while only 68 columns are needed in the ColMap representation.

Because this example is somewhat contrived, and real-world maps are much more complicated, a statistical analysis of several existing datasets is provided to demonstrate the potential advantage of the column-based representation. We then present a direct comparison of both memory usage and performance for these datasets. A new dataset, obtained by a laser scanner mounted on-board a small hexacopter UAS, is also used for comparison and made available for public use. A ROS implementation of the ColMap software is also made available in the Supplementary Materials.

2. Description of the algorithm

2.1. Data structure

The basic data structure is illustrated in Fig. 2. Three-dimensional space is divided into cells (voxels), and each cell is given an occupancy probability, as per the standard BOG technique. Each vertical column of the OG (hereafter referred to as a leaf) is stored as a leaf header, followed by a list of probability values. This list can either be uncompressed or compressed using a simple run-length encoding technique. It is this compression along the vertical direction that allows the algorithm to take advantage of the high degree of correlation often present between vertically adjacent cells, without regard to their horizontal correlation. The leaf header stores its compressed/uncompressed status as well as the vertical location of the first list element (for a compressed leaf) or the length of the list (for an uncompressed leaf); see Fig. 3(a). Storing the length of the list allows the grid to grow dynamically in the vertical direction.

Probabilities are stored as either 14-bit fixed-point, 30-bit fixed-point, or 30-bit floating-point values. The reason for these non-standard representations is that two bits are reserved for special flags, as illustrated in Fig. 3(b). The probability value plus the flags thus occupy either 16 or 32 bits. Hereafter, these will be referred to as 16-bit mode and 32-bit mode. Justification for this reduced resolution shall be provided in a later section.

Compressed leaves are stored as a list of elements, in which each element represents either the probability value of a single cell or a sequence of identical probability values. These two options are distinguished by the least significant bit of the element (Fig. 3(c)). A standard run element consists of a run length followed by a probability value. Commonly occurring probability values (i.e., maximum, minimum, and initial) are stored more efficiently by utilising type bits instead of specifying the full probability value. This is depicted in Fig. 3(c) for the 16-bit mode, where it is shown that a standard run element occupies 32 bits, whereas a special run element occupies only 16. This can be considered a very simple type of entropy encoding. The final element in a compressed node is indicated by the “end-of-list” bit.

Uncompressed leaves are stored as a series of cell probabilities in contiguous memory. In this case, the two reserved bits are used for update flags, which indicate whether a cell has been traversed (without a return) or hit by a laser scan (Fig. 3(d)). These are important in implementing the update technique discussed in a later section.

The horizontal part of the grid is stored as an N^2 tree (Fig. 2), with branches and leaves being initialised when they are first sensed during a scan. The reason for this is primarily to enable efficient dynamic growth of the map and to avoid storing leaf pointers for regions of the map that remain unexplored. This is

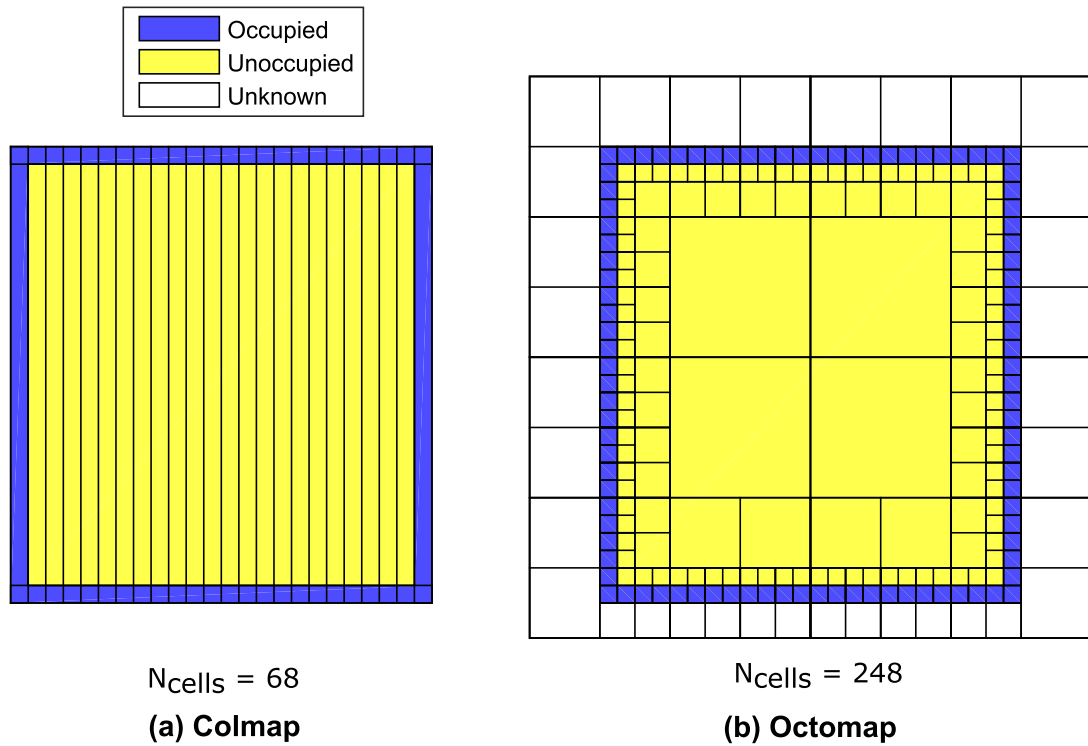


Fig. 1. Simplified comparison of the storage methods of ColMap and Octomap, using the example of an empty room (cross-section, side view).

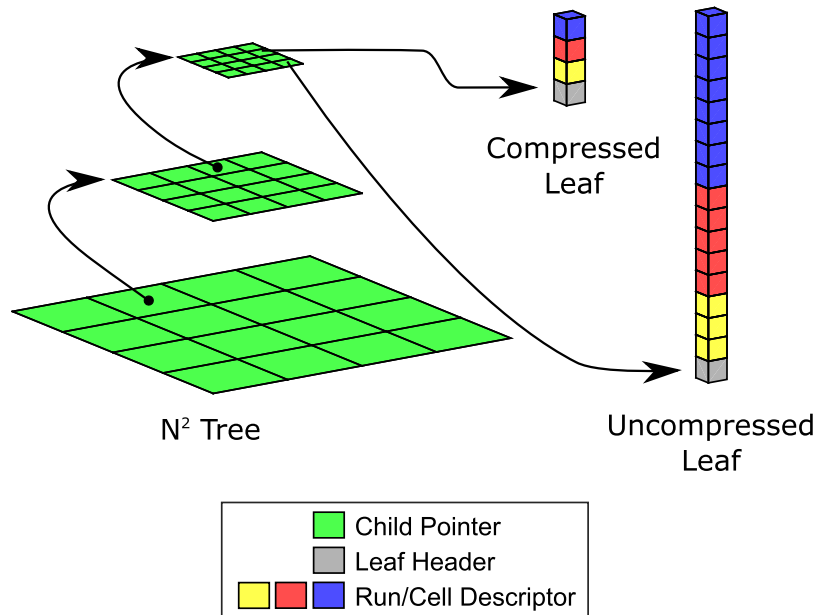


Fig. 2. Outline of ColMap's basic data structure.

effective in situations where exploration takes place mainly in the horizontal plane, which is typical of most (but not all) robot missions. Other tree-based implementations are motivated by the ability to perform multi-resolution queries; but this is not the case with ColMap. That being said, while the current implementation of the tree consists purely of an array of child pointers (with a null pointer indicating an uninitialised child), the tree nodes can be used to store additional metadata about their leaves to aid in multi-resolution queries. For example, maximum likelihood

probabilities at several horizontal planes can be stored and updated in real-time to avoid querying every leaf when performing predominantly-2D path planning and navigation tasks.

It is well known that variable-length lists, such as those used for the nodes here, are not particularly efficient for insertion, deletion, or access. However, in comparison to a purely tree-based method such as Octomap, this loss of efficiency is partially negated by the improved locality of reference and the (in general) limited length of the list. Performance on these tasks is comparable to Octomap and is discussed in Section 5.

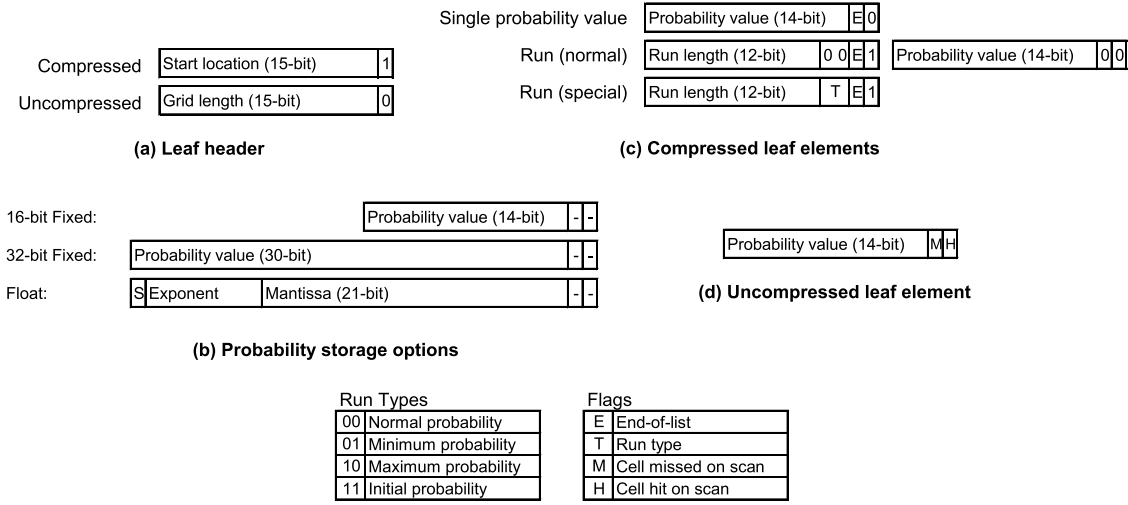


Fig. 3. Memory layout of the various elements.

2.2. Probability updates

Upon reception of a measurement, the new occupancy probability of each cell can be updated according to Bayes' rule [6]:

$$P(x | z_{1:t}) = \frac{P(z_t | x)P(x | z_{1:t-1})}{P(z_t | x = 1) \cdot P(x | z_{1:t-1}) + P(z_t | x = 0) \cdot (1 - P(x | z_{1:t-1}))} \quad (1)$$

where x is the occupancy status of the cell (occupied or not-occupied), and the measurement, z_t , is either occupied (where a reflection is returned) or unoccupied (where the beam passes through the cell). Updating cells that are detected as unoccupied is an important feature of this type of algorithm, as empty space is thus captured and represented also. A ray-tracing algorithm [19] is used to trace the sensor ray through the OG and thus to determine which cells to update.

The log-odds representation, L , is stored in lieu of the actual probability value:

$$L(x) = \log \left(\frac{P(x)}{1 - P(x)} \right) \quad (2)$$

When substituted into (1), this obviates the need to calculate the normalising constant and removes all multiplication and division operations, resulting in the update equation:

$$L(x | z_{1:t}) = L(x | z_{1:t-1}) + \log \left(\frac{P(z_t | x = 1)}{P(z_t | x = 0)} \right) \quad (3)$$

Applying Bayes' rule to the final term in isolation yields:

$$L(x | z_{1:t}) = L(x | z_{1:t-1}) + L(x | z_t) \quad (4)$$

Either (3) or (4) can be used, the difference being that (3) uses the forward sensor model, whereas (4) uses the inverse sensor model. In both cases, updating the probability is a matter of incrementing or decrementing the log-odds value by a pre-determined amount.

Following Hornung et al. [5], we “clamp” the log-odds values to certain limits to avoid over-confidence in a particular cell's occupancy status. This helps in particular with dynamic environments where a cell's actual occupancy status may change over time. It is also critical to achieving compression, as the tendency of each cell's probability to tend to a constant value over time is what allows the run-length coding to be effective. We also incorporate the “single-update-per-scan” policy of Hornung et al. [5], which improves the accuracy of the map when utilising

data from a scanning laser device. If a cell is traversed by multiple laser beams in a single scan, its probability is only updated once. Positive detections take precedence over negative ones. This is the reason for the “hit” and “miss” flags in the uncompressed leaf representation — these flags are updated first (which may happen multiple times in a single scan), then at the end of the scan the probability values are updated accordingly. The flags are then reset, ready for the next scan.

Several algorithms to update a compressed leaf without full decompression were devised; however, their complexity resulted in their not being significantly faster than the naïve method (i.e., decompress the leaf, update the probabilities, and recompress the leaf). Recompression is performed only at the end of each scan, which provides a performance increase at the expense of a transient increase in memory consumption. However, performance on scan insertion remains superior to that of Octomap (at least for the 16-bit mode; 32-bit mode was not benchmarked), as is discussed in Section 5.

2.3. Reduced-resolution storage of probability values

Most previous mapping algorithms store probability (or log-odds) values as 32-bit floating point numbers. However, such a high resolution is rarely needed when the clamping technique is used to limit the range of possible probability values. For example, with the clamping range recommended by Hornung et al. [5] for typical sensors ($L_{min} = -2.0$, $L_{max} = 3.5$), log-odds values can be stored as 16-bit integer values with a resolution of 0.000084. This is more than adequate given the suggested additive updates of 0.85 and -0.41 for a hit and miss, respectively. The corresponding probability resolution varies due to the non-linearity of the log-odds transformation, between a minimum of 0.0000088 and a maximum of 0.000021.

In the default implementation of ColMap, we represent the log-odds value as a signed, 14-bit fixed-binary-point value. The use of fixed-binary-point values accelerates conversion back to the floating point representation, and preserves the zero value for an unknown cell ($p = 0.5$), at the cost of a slight reduction from the theoretical maximum resolution. For the default log-odds range, this gives a resolution of 0.00098, or in terms of occupancy probability, between 0.00010 and 0.00025. This is more than sufficient for the present application. All updates to the map are done in the scaled representation so there is no run-time penalty for using this representation, in fact it is faster due to the use of integer arithmetic. As mentioned earlier, 30-bit fixed- and

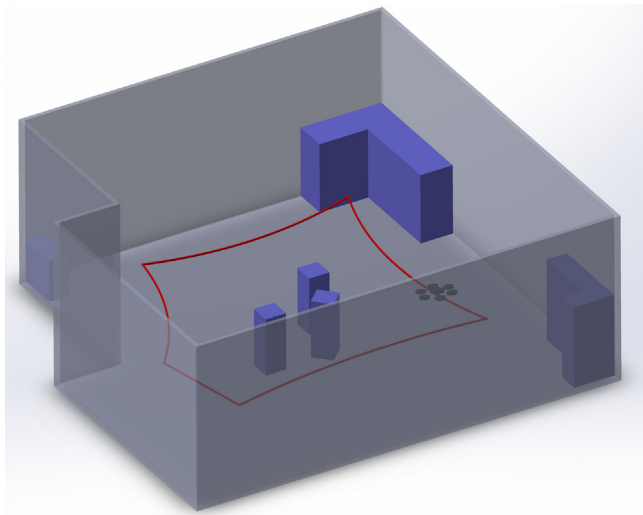


Fig. 4. Sketch of flight arena used for DST dataset capture, showing room (grey), obstacles (blue), approximate flight path (red), and UAS (black). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

floating-point values can also be utilised, however we have found this to be unnecessary.

3. The DST dataset

In addition to testing ColMap on existing datasets, we make available a new dataset called “DST”. The DST dataset was recorded using a Velodyne Puck™ LIDAR mounted on a hexacopter unmanned aerial system (UAS). The drone’s movement was tracked using a system of 48 Optitrack motion-capture cameras, allowing the origin point and orientation of the sensor scans to be determined.

The UAS was flown around an indoor flight arena 10 m wide x 15 m long x 8 m high, which contained several obstacles. Data was taken during ten flights, with the UAS being automatically commanded to follow the same pre-determined path each time. This makes it possible to test the repeatability of maps produced from these scan data, either by “pure-mapping” methods (e.g. ColMap, Octomap) or a SLAM algorithm. The layout of the room, the path of the UAS, and the resulting maps are illustrated in Fig. 4. ROSBag files containing the raw scan and pose data, as well as Octomap files containing the generated maps at two different resolutions (0.05 m and 0.1 m) are available as Supplementary Materials.

4. Analysis of memory consumption

4.1. Statistical analysis of existing and DST datasets

Any compression method takes advantage of the lack of information entropy in the data. Octree methods exploit the fact that cubic volumes (larger than the cubes that make up the grid itself) are likely to have the same occupancy status. The hypothesis behind the present approach is that vertically-adjacent cells are more likely to have the same occupancy status than are horizontally adjacent cells. To test this hypothesis, we analysed the datasets detailed in Table 1.

Three-dimensional views of the existing datasets are shown in Fig. 5, and the results of the statistical analysis are shown in Table 2 for both the original and maximum-likelihood versions of the maps. Values for the DST dataset are an average across the

ten data-collection flights. The “proportion of leaves hit” column indicates the percentage of vertical columns in the full-grid map that were touched at least once by a sensor scan. This includes both a positive return, as well as the sensor beam passing through that column. A value well below 100% indicates that ColMap’s use of the tree structure for the 2D grid will potentially result in a memory saving, because memory need not be allocated for leaves that have never been hit. The values are, however, dependent on the type of environment mapped. For areas with large occluded regions the value is quite low (e.g., New College, 31%); while for a single, essentially rectangular room that can be completely explored (DST), the value is close to 100%. The values for the other maps are somewhere in between (roughly 50%).

Note that the *minimum* bounding boxes of the existing datasets have been used in calculating the total number of possible columns, which results in the highest percentage-of-nodes-hit value. The real benefit of the tree representation is where the bounding box is not known a priori, and so has to be set to a very large value. In this case the percentage of nodes hit would be far less, as the robot is likely to have only explored a small region of the large bounding box. In most cases the bounding box is likely to be much larger in the horizontal direction than in the vertical direction, as the robot explores mainly in the horizontal direction. This justifies the use of the tree representation in only the horizontal direction, as opposed to all three directions.

In ColMap, the majority of the compression is achieved by the run-length encoding of the probability values within the leaves. Potential compression ratios are indicated by the average number of runs per leaf, and the ratio of grid vertical size to average runs per leaf (shown in Table 2). A “run” is defined as one or more vertically-consecutive identical probability values. Note that these values include both free and occupied space. A smaller number of runs per leaf indicates that the probability values can be more effectively compressed by run-length encoding. Comparing this to the grid vertical size gives an idea of the compression ratio achievable. For example, if the average number of runs were equal to the vertical grid size, then no compression is achievable.

The ratios of grid vertical size to average number of runs per leaf in the datasets analysed here indicate that significant compression is achievable. (Note that these do not directly represent the compression ratio, as there is some overhead in the run-length representation.) In particular, the values for the maximum likelihood maps justify the observation made earlier that most leaves in a converged map are likely to consist of only a few runs (e.g. some structure on the ground, followed by free space above, followed by unknown space above that). The ratio for a particular map will depend on a number of factors, including:

- The complexity of the structures present;
- The extent to which the map has converged to the minimum and maximum probability values;
- The coverage of the sensor scans (because sensors that leave “holes” in the scanned volume will degrade the compression);
- The sensor $P(\text{hit}|\text{occupied})$, where a less certain sensor will introduce a higher level of entropy into the map before converging to the clamped values.

Examples of the efficient storage of certain features can be seen in Fig. 6, in which the occupied runs of the maximum likelihood map are shown, coloured by height for clarity. Fig. 6(a) is a close-up of the Freiburg campus dataset, showing a lamp post and a tree. These structures are stored quite efficiently, as is discernable by the fact that they contain a significant number of vertical runs of occupied cells. The ground is also stored efficiently, as

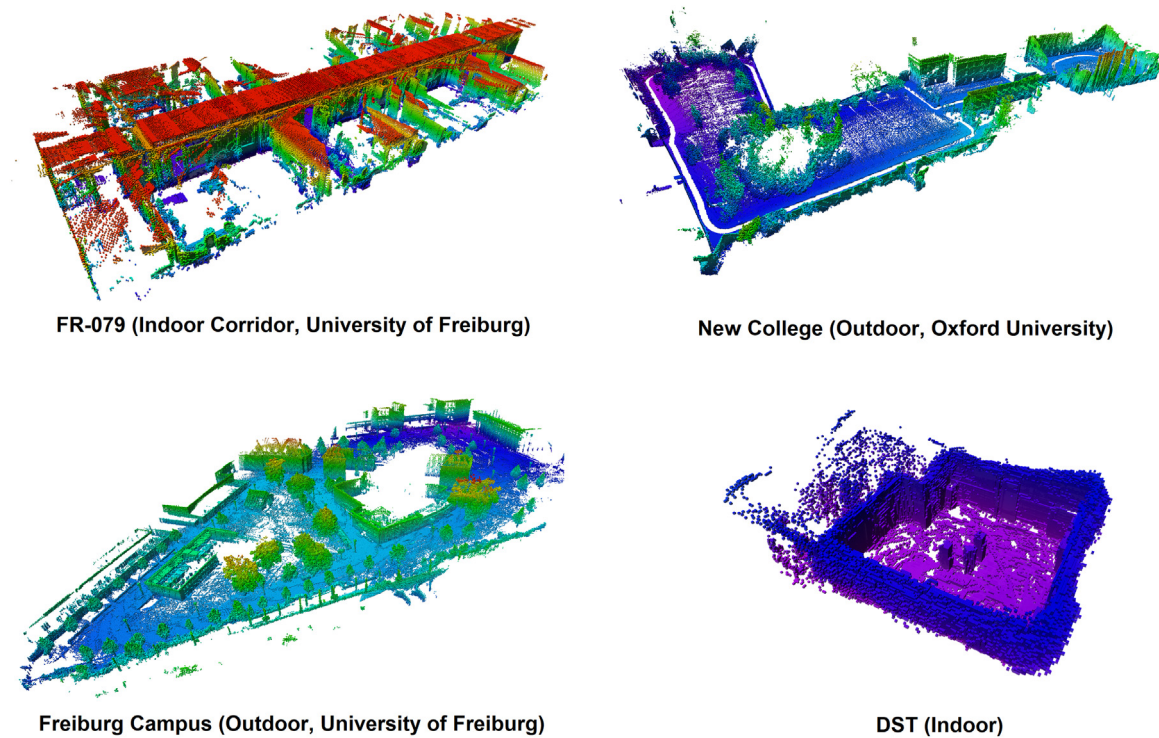


Fig. 5. 3D views of the datasets used to analyse ColMap (renderings of existing datasets are from [5]).

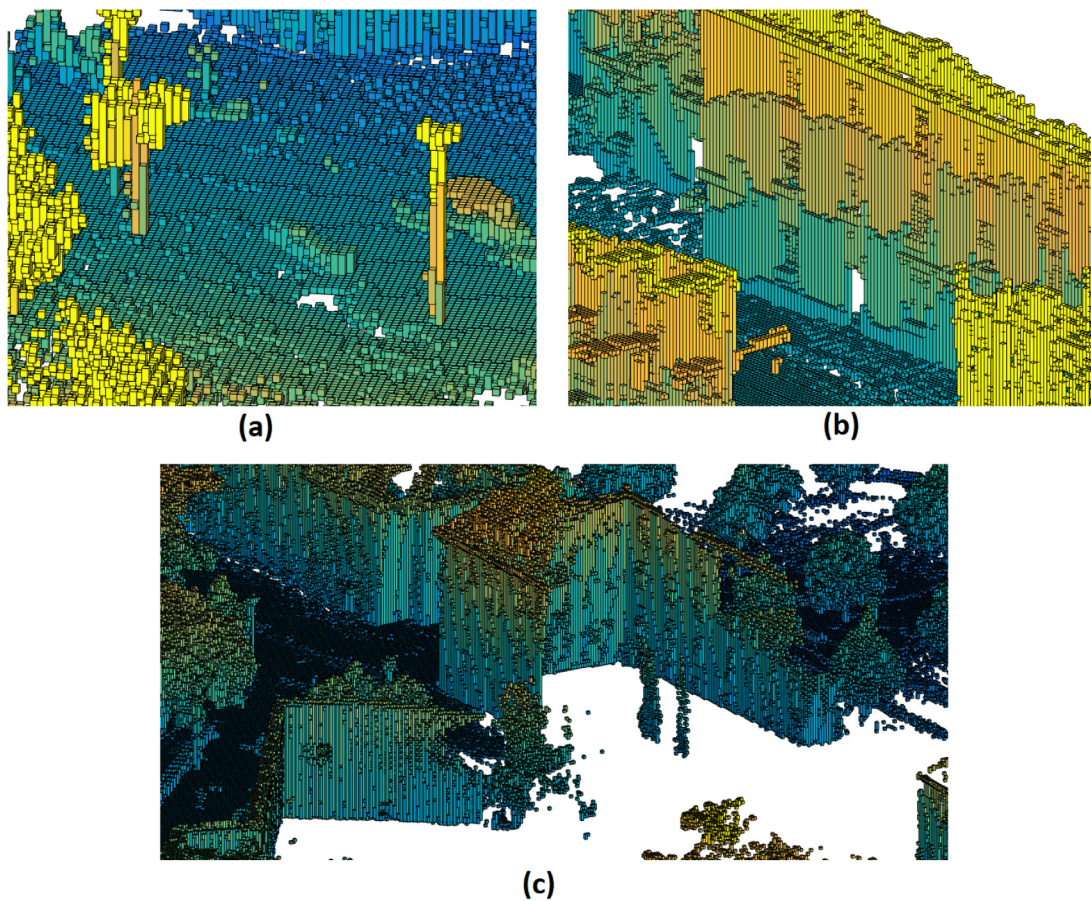


Fig. 6. Close-up view of certain features within ColMaps based on existing datasets.

Table 1
Details of datasets used to demonstrate the performance of ColMap.

Name	Description	Ref.	Extent	Grid Resolution	Sensor Type
FR-079	Indoor, corridor	[20]	43.8 m × 19.2 m × 3.3 m	0.05 m	SICK LMS laser scanner
Freiburg campus	Outdoor, university campus	[21]	292 m × 167 m × 28 m	0.2 m	SICK LMS laser scanner
New College (Epoch C)	Outdoor, university campus	[22]	250 m × 161 m × 33 m	0.2 m	2x SICK LMS laser scanner
DST	Indoor, room with arbitrary obstacles	N/A	12 m × 12 m × 7 m	0.1 m	Velodyne Puck

Table 2
Dataset statistics pertinent to the compression performance of ColMap.

Name	Grid size (cells)	Number of cells	Proportion of leaves hit	Map with full probability values		Maximum likelihood map	
				Average runs per leaf	Ratio of vertical size to avg runs per leaf	Average runs per leaf	Ratio of vertical size to avg runs per leaf
FR-079	873 × 363 × 80	25,351,920	50.9%	9.8	8.12	4.35	18.41
Freiburg campus	1462 × 835 × 144	175,790,880	52.5%	8.4	17.16	3.61	39.84
New College (Epoch C)	805 × 1248 × 176	176,816,640	31.4%	6.3	27.93	3.48	50.59
DST	121 × 120 × 70	1,016,400	90.2%	10.3	6.84	4.58	15.37

columns where no structures are present usually consist of a single occupied cell for the ground, followed by a long run of unoccupied cells that represent the free space above. This makes these areas very efficient to both store and access in compressed form, as they essentially form a height map.

Fig. 6(b) is a close-up of a building wall in the New College map, where efficient storage is indicated by the relatively large height of the runs. Note that the walls are not perfectly aligned with the vertical direction of the map, possibly due to a slight error in the pose estimate of the laser scanner. Ideally the wall would consist of single run of occupied cells, however significant compression is still achieved despite the slight tilt of the wall. Fig. 6(c) shows several walls that are not aligned with the horizontal axes in the Freiburg Campus map, for which there is no penalty in the ColMap representation, except for some additional “quantisation noise” due to the finite resolution of the map. This is in contrast with both Octomap and RMAP, which suffer an inherent penalty for structures that are not aligned with the horizontal axes.

4.2. Comparison to existing algorithms

In this section, memory usage for ColMap is compared with that for a full OG and for an Octomap [5]. The N^2 tree in ColMap has N set to 16, which we found to be a good balance between speed and memory overhead. Pointers in both ColMap and Octomap are assumed to be 64 bits (note that this is the reason why memory usage statistics for the pre-existing datasets do not match the values of Hornung et al. [5]; the latter used a 32-bit machine). The assumption of a 64-bit machine is justified as most newer machines are of this nature, and also a 64-bit address space is necessary to access more than 4GB of memory. Memory usage for ColMap is shown for three states after insertion of all scans from the relevant dataset into the map:

1. All nodes uncompressed;
2. All nodes compressed;
3. All nodes converted to maximum-likelihood representation and compressed.

Octomap memory usage is given for: the full tree (equivalent to ColMap’s fully uncompressed condition), the pruned tree (equivalent to ColMap’s fully compressed condition), and the pruned maximum-likelihood representation (equivalent to ColMap’s

maximum-likelihood fully compressed condition). These were obtained using the `OcTree::memoryUsage()` function in the Octomap API. To ensure that the maps ingested by the compression algorithms were identical, probability values were directly exported using the Octomap API and then imported into ColMap. This is in contrast to inserting the scan data into each software package separately, which may result in slightly different maps due to possible differences in their scan-insertion algorithms.

Because ColMap uses 16-bit probability values by default, whereas Octomap uses 32-bit values, memory usage is also shown for the 32-bit-probability version of ColMap. Also, because the size of an uncompressed ColMap node depends on the pre-defined vertical extent of the map, “tailoring” the vertical extent to the map data would not result in a fair comparison. Thus, the vertical size of the ColMap map is rounded up to the next-highest multiple of 16. This same vertical size is used when calculating the full-grid memory usage.

Memory usage comparisons are shown in Table 3 for both 16- and 32-bit probability values. These show that in all cases, the ColMap representation outperforms the full grid even when the ColMap is left fully uncompressed. The saving of the uncompressed ColMap over the full grid relates directly to the “proportion of nodes hit” statistic discussed previously. Memory usage is between 4.2 and 13.6 times lower for the compressed ColMap, compared to the uncompressed one, for 16-bit probabilities. For 32-bit probabilities, the compression ratio is higher (5.7x compared with 21.0x). This is due to the fact that commonly occurring probability values (minimum, maximum, and unknown) are stored as bit fields within the run headers; hence, the memory penalty changing from a 16-bit representation to a 32-bit representation is not simply double, but closer to 1.4x, for the compressed ColMap. Compression ratios for the maximum-likelihood map over the uncompressed map range from 7x to 19x for the 16-bit probability case, and from 13x to 37x for the 32-bit case.

The memory usage of ColMap also compares favourably to Octomap. For the 32-bit probability case, the uncompressed ColMap is about on par with the full Octomap for the sparser maps (Freiburg campus and New College). For the more dense maps (FR-079 and DST), ColMap outperforms Octomap due to the overhead of storing many levels of inner-node pointers in the latter. Both methods achieve much lower compression relative to the full grid for the denser maps, which is to be expected.

Table 3

Comparison of memory usage to Octomap, when 16-bit probabilities are used for the full-grid and ColMap. Comp – compressed, ML – maximum likelihood.

Name	Memory usage (MB)		ColMap (16-bit)			ColMap (32-bit)			Octomap		
	Full grid (16-bit)	Full grid (32-bit)							Full	Pruned	ML Pruned
			Uncomp	Comp	ML Comp	Uncomp	Comp	ML Comp			
FR-079	48.54	97.09	26.70	6.09	3.32	51.43	8.56	3.32	147.10	83.25	49.45
Freiburg campus	335.93	671.85	183.65	21.21	11.54	360.19	29.88	11.54	375.86	260.48	148.26
New College (Epoch C)	337.94	675.88	109.66	8.09	5.78	215.63	10.27	5.78	182.68	101.15	71.90
DST	1.90	3.81	1.89	0.45	0.28	3.61	0.63	0.28	13.55	5.18	3.73

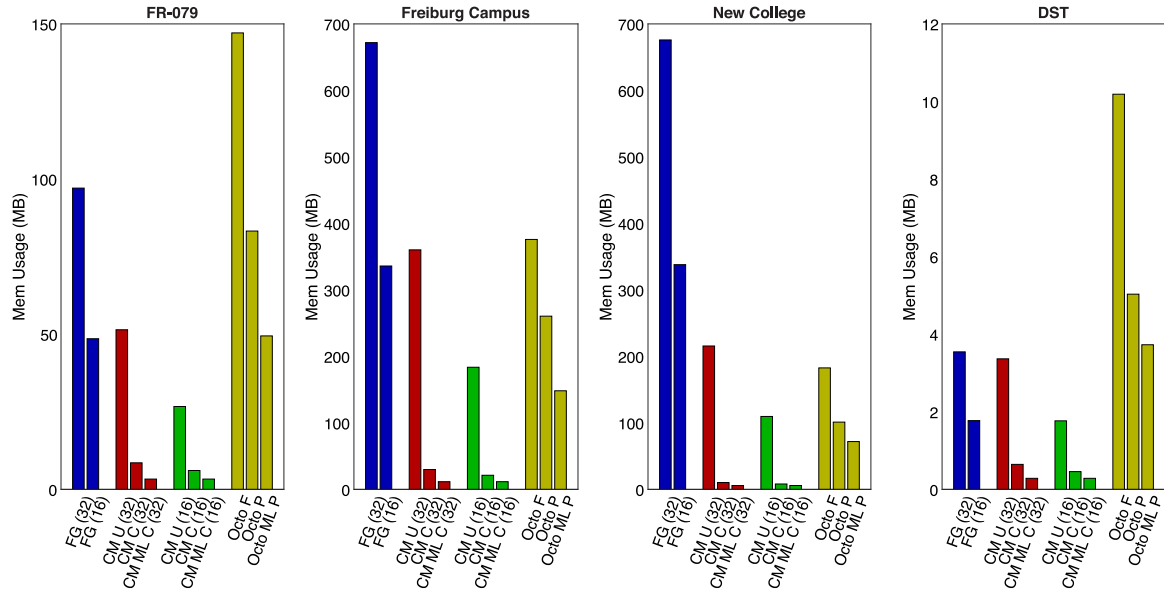


Fig. 7. Memory usage for ColMap (CM), Octomap (Octo), and full-grid (FG) representations for each mapping-data set. U – uncompressed, C – compressed, F – fully expanded, P – pruned.

Comparing the compressed ColMap to the “compressed” (i.e., pruned) Octomap shows that memory usage is consistently

around 12x lower when 16-bit probabilities are used and around 9x lower with 32-bit probabilities. Comparing the maximum-likelihood compressed/pruned maps reveals a consistent 13–15x reduction in memory usage across all cases. Note that there is no penalty for going from 16-bit to 32-bit probabilities for the maximum-likelihood compressed ColMap, because of the efficient representation of commonly occurring values. A graphical summary of memory usage comparisons is shown in Fig. 7.

It must be pointed out that the size of the structures ColMap uses are variable, as opposed to Octomap’s, which are fixed in size. Depending on the memory allocation algorithm used, this may result in some additional memory wastage due to fragmentation. However, with the sophisticated algorithms present in today’s operating systems, this is unlikely to be significant. Another issue is the fact that while inserting a new scan, ColMap decompresses the relevant areas temporarily, leading to a transient increase in memory consumption. If a single “scan” consists of a large number of points, then this increase can be significant. To demonstrate, memory consumption is plotted against time in Fig. 8. It can be seen that the peak (transient) memory consumption is significantly higher than that of the static map for the FR-079 and Freiburg Campus datasets, but is negligibly higher for the New College dataset. The reason for this is that the FR-079 and Freiburg Campus datasets contain relatively few “scans”, each containing a large number of points, whereas the New College dataset contains many scans with relatively few

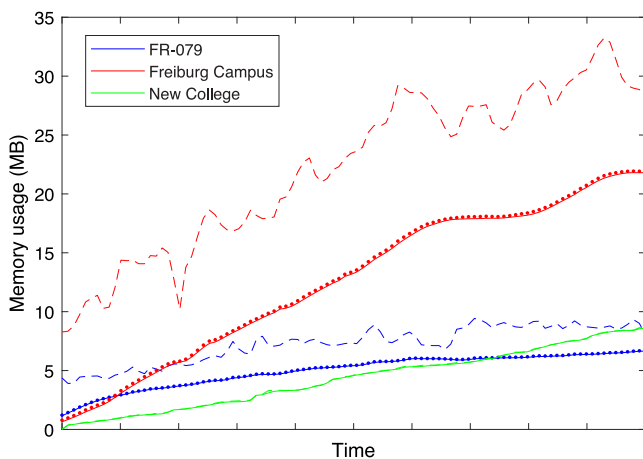


Fig. 8. Memory consumption vs time, as scans are inserted into the map. Shown are: static memory usage (—), peak memory usage (---), and peak memory usage after subdividing scans to match processing time of Octomap (•). Note that for the New College dataset, peak memory usage differs very little from the static value.

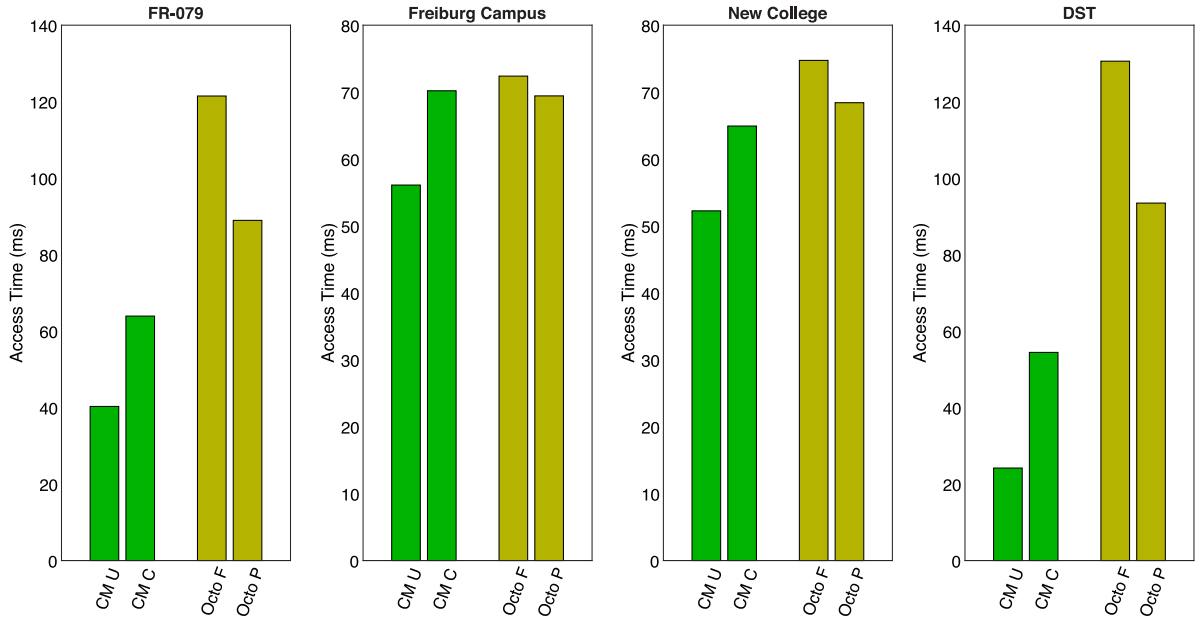


Fig. 9. Time to access one million random cells.

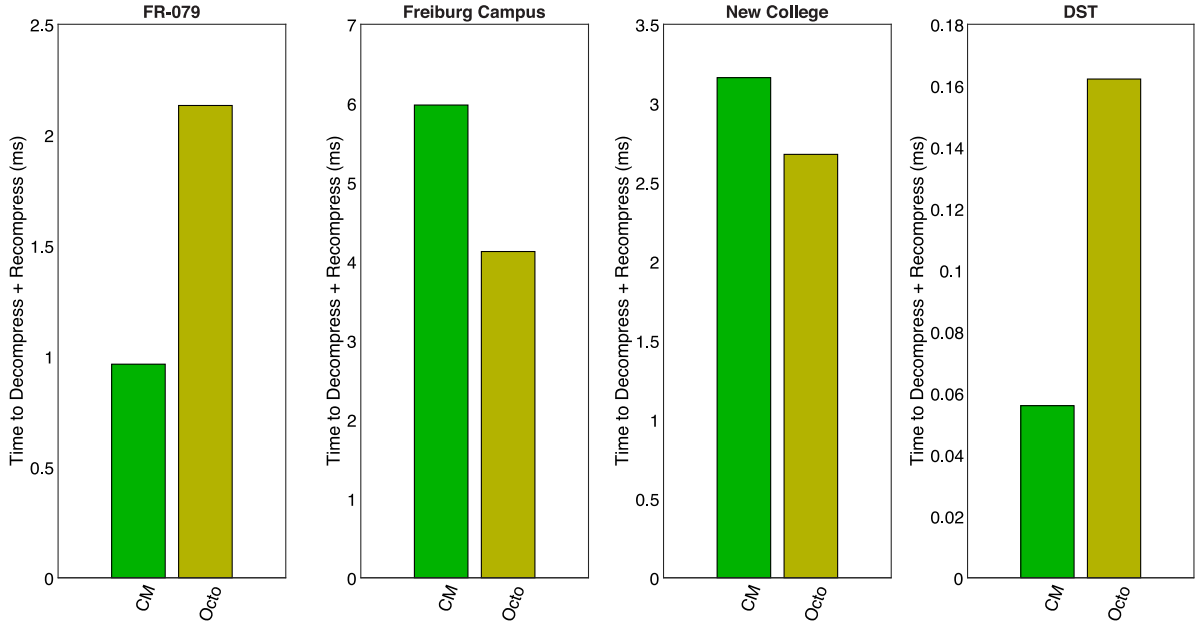


Fig. 10. Time taken to decompress and recompress entire map.

points per scan. This suggests a simple way of reducing the transient memory increase — divide a large scan up into smaller batches, and recompress the map after inserting each batch. This of course, comes at the cost of increased processing time, which is evident in Fig. 11 where it can be seen that the New College dataset takes much longer to process than the other datasets. To arrive at a fair comparison with Octomap, we determined the batch size necessary to bring the total processing time into line with that of the latter (since, by default, ColMap is about four times faster than Octomap at processing scans). This batch size turned out to be quite small, equivalent to dividing each scan up into approximately 60 batches, for both the FR-079 and Freiburg Campus datasets. The resulting transient memory increase then becomes negligible, as shown in Fig. 8.

By choosing how to divide up each scan into smaller batches, the user can easily perform a tradeoff between speed and peak

memory consumption that best suits their system. It is also worth noting that the *relative* magnitude of the transient memory increase (i.e. as a fraction of the static usage) will become smaller as the map grows, since this increase is proportional to the fraction of the map that the sensor can “see” at any given time. We theorise that as the map becomes large enough to push the memory limits of the system, the “visible” fraction of the map should decrease, making this transient increase less important.

5. Analysis of performance

In order for ColMap to be useful, it must be shown that the benefit of lower memory consumption is not offset by a significant additional computational burden. In this section, the time taken to perform the following operations is measured and compared with that of Octomap:

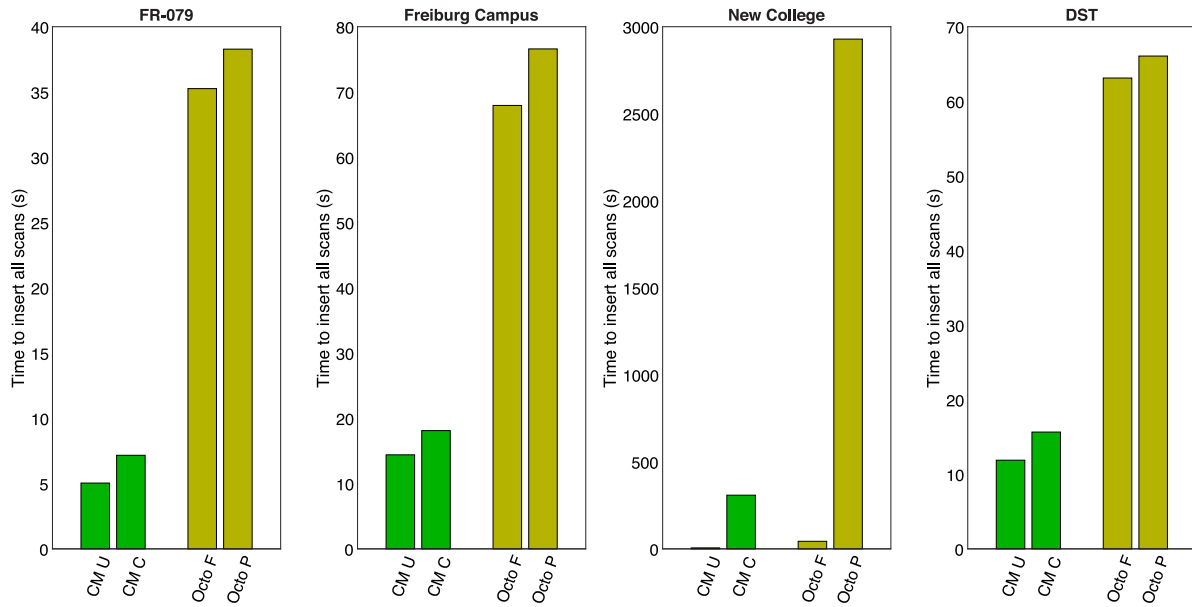


Fig. 11. Time taken to generate map from scratch by inserting all scans.

- Access probability values within the map;
- Compress/decompress the map;
- Insert new scans into the map.

All benchmarking was done using a single core on an Intel i7-5500U 2.4 GHz CPU, running LUbuntu 18.04. All operations were performed using the native APIs, as opposed to the ROS subsystem. The 16-bit version of ColMap was used, and the times reported are actual CPU times spent in each process, as opposed to wall-clock time. The time to read probability values from one million cells randomly distributed across the extent of the map is shown in Fig. 9. As expected, the efficiency of accessing an uncompressed ColMap is slightly better than accessing the compressed map, due to the overhead of processing the run-length encoded leafs. In contrast, accessing an uncompressed Octomap is slightly less efficient than accessing the compressed Octomap, due to the overhead of having to traverse (on average) a deeper tree. For sparse maps (Freiburg Campus and New College), access times for the two methods are comparable, with ColMap being slightly more efficient. For the denser maps, ColMap significantly outperforms Octomap.

The time taken to compress and then decompress the entire map is shown in Fig. 10. Here, Octomap outperforms ColMap for the sparse maps, while the opposite occurs for the dense maps. Entire map compression/decompression would rarely be performed in real-time operation, but the benchmarks are shown here for completeness. A better benchmark is the time taken to insert scans into the map, as would occur during real-time mapping. The time taken to construct a map from scratch by inserting all scans is shown in Fig. 11. Values are shown for both an uncompressed map and a compressed map, in the latter case of which compression is performed at the end of each scan insertion. Under all conditions ColMap significantly outperforms Octomap, by an average factor of about 4x. Note that the large discrepancy between the compressed and uncompressed insertion times for the New College map is due to the fact this dataset contains a large number of scans consisting of relatively few points per scan. This is in contrast to the other datasets, which contain a smaller number of scans with a relatively large number of points per scan. Thus, compression is performed much more often for the New College dataset, resulting in much longer insertion

times. It should also be noted that the speed of ColMap, in the default scenario of recompressing once after each scan insertion, does come at a cost of increased transient memory consumption. However, a tradeoff can be performed by the user as discussed in Section 4.2.

6. Conclusions & future work

Mobile robots (in particular those that are airborne) possess limited computing resources. Operating within memory and CPU constraints is thus important for any algorithm/software used thereon. New mapping software for mobile robots has been proposed, which combines ideas and benefits of previous algorithms to produce a result that is superior in terms of memory consumption and computational burden:

- A tree representation for efficient dynamic growth of the map, as pioneered by Octomap [5] and expanded on by RMAP [8];
- Explicit representation of unexplored space within the map;
- Compression via fusion of cells that are not necessarily cubic, in a similar way to the axis-aligned features used in RMAP [8];
- Exploitation of the verticality of the built and natural world, in a manner inspired by previous 2.5D approaches [13,15];
- Maintaining of the occupancy grid representation for use in path planning [5,8].

Evaluation of the software on both new and existing datasets showed that memory consumption was reduced by an order of magnitude with respect to an existing comparable OG compression algorithm [5], while memory consumption relative to the full OG is reduced by a factor of between 20 and 40. Benchmarks showed that this memory saving comes with no performance penalty in either map creation or map query.

With further work, ColMap could be tailored to be more efficient for specific mission scenarios. Ideas being explored include:

- The use of a tail pointer in each leaf to make top-down leaf traversal possible. This may provide increased efficiency in scenarios in which an airborne robot is navigating from above structures and is thus interested in the highest obstacle at any particular location.

- The combination of multiple ColMaps to deal with structures like multi-storey buildings, where it becomes inefficient to store data from every storey in a single ColMap leaf.
- The storage of certain properties of each leaf in the N^2 tree structure to make accessing them more efficient. For example, maximum-likelihood probabilities at several horizontal planes could be stored in the tree and updated in real-time to avoid having to query every leaf when performing predominantly 2D path planning and navigation tasks.
- Delayed-compression techniques in which map compression is performed not after every scan, but at a lower rate, to reduce computational burden. One could, for example, perform compression only when memory usage approaches a certain limit, thus optimising the memory vs speed tradeoff.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research was made possible by support and funding provided by Defence Science and Technology (DST), Australia, part of the Australian Department of Defence.

References

- [1] R.B. Rusu, S. Cousins, 3D is here: Point Cloud Library (PCL), in: IEEE International Conference on Robotics and Automation, Shanghai, China, 2011, pp. 1–4, <http://dx.doi.org/10.1109/ICRA.2011.5980567>.
- [2] M.A. Fischler, R.C. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, *Commun. ACM* 24 (6) (1981) 381–395, <http://dx.doi.org/10.1145/358669.358692>.
- [3] H. Moravec, A. Elfes, High resolution maps from wide angle sonar, in: Proceedings of the IEEE International Conference on Robotics & Automation, ICRA, St. Louis, USA, 1985, pp. 116–121.
- [4] A. Elfes, Using occupancy grids for mobile robot perception and navigation, *Computer* 22 (6) (1989) 46–57, <http://dx.doi.org/10.1109/2.30720>.
- [5] A. Hornung, K.M. Wurm, M. Bennewitz, C. Stachniss, W. Burgard, OctoMap: an efficient probabilistic 3D mapping framework based on octrees, *Auton. Robot.* 34 (2013) 189–206, <http://dx.doi.org/10.1007/s10514-012-9321-0>.
- [6] S. Thrun, W. Burgard, D. Fox, R.C. Arkin, *Probabilistic Robotics*, MIT Press, Cambridge, MA, USA, 2020.
- [7] S. Thrun, Learning occupancy grids with forward models, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, Maui, HI, USA, 2001, pp. 1676–1681, <http://dx.doi.org/10.1109/IROS.2001.977219>.
- [8] S. Khan, A. Dometios, C. Verginis, C. Tzafestas, D. Wollherr, M. Buss, RMAP: a rectangular cuboid approximation framework for 3D environment mapping, *Auton. Robot.* 37 (2014) 261–277, <http://dx.doi.org/10.1007/s10514-014-9387-y>.
- [9] E. Einhorn, C. Schröter, H.-M. Gross, Building 2D and 3D adaptive-resolution occupancy maps using Nd-Trees, in: 55th International Scientific Colloquium, Germany, 2010, pp. 306–311.
- [10] C. Parra, R. Murrieta-Cid, M. Devy, M. Briot, 3D modelling and robot localization from visual and range data in natural scenes, in: 1st International Conference on Computer Vision Systems, ICVS, 1999, pp. 450–468.
- [11] S. Singh, A. Kelly, Robot planning in the space of feasible actions: two examples, in: IEEE International Conference on Robotics and Automation, 1996.
- [12] A. Souza, L.M.G. Goncalves, Occupancy-elevation grid: an alternative approach for robotic mapping and navigation, *Robotica* 34 (11) (2016) 2592–2609, <http://dx.doi.org/10.1017/S0263574715000235>.
- [13] R. Triebel, P. Pfaff, W. Burgard, Multi-level surface maps for outdoor terrain mapping and loop closing, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, Beijing, 2006, pp. 2276–2282, <http://dx.doi.org/10.1109/IROS.2006.282632>.
- [14] P.-M. Victor, J. Cabrera, A. Domínguez Brito, D. Hernandez-Sosa, J. Isern, E. Fernandez Perdomo, Efficient plane detection in multilevel surface maps, *J. Phys. Agents* 5 (2011) <http://dx.doi.org/10.14198/JoPha.2011.5.1.03>.
- [15] I. Dryanovski, W. Morris, J. Xiao, Multi-volume occupancy grids: An efficient probabilistic 3D mapping model for micro aerial vehicles, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, Taipei, 2010, pp. 1553–1559, <http://dx.doi.org/10.1109/IROS.2010.5652494>.
- [16] Helen Oleynikova, A. Millane, Z. Taylor, E. Galceran, J. Nieto, R. Siegwart, Signed distance fields: A natural representation for both mapping and planning, in: Workshop on Geometry and Beyond, 2016.
- [17] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, J. Nieto, Voxblox: incremental 3D euclidean signed distance fields for on-board MAV planning, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, Vancouver, BC, pp. 1366–1373, <http://dx.doi.org/10.1109/IROS.2017.8202315>.
- [18] V. Usenko, L. von Stumberg, A. Pangercic, D. Cremers, Real-time trajectory replanning for MAVs using uniform B-splines and a 3D circular buffer, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, Vancouver, BC, 2017, pp. 215–222, <http://dx.doi.org/10.1109/IROS.2017.8202160>.
- [19] J. Amanatides, A. Woo, A fast voxel traversal algorithm for ray tracing, in: Proceedings of Eurographics, Amsterdam, The Netherlands, 1987.
- [20] A. Hornung, OctoMap 3D scan dataset, 2020, <http://ais.informatik.uni-freiburg.de/projects/datasets/octomap/>. (Accessed 1 September 2020).
- [21] B. Steder, Freiburg campus 360 degree 3D scans, 2020, <http://ais.informatik.uni-freiburg.de/projects/datasets/fr360/>. (Accessed 1 September 2020).
- [22] M. Smith, I. Baldwin, W. Churchill, R. Paul, P. Newman, The new college vision and laser data set, *Int. J. Robot. Res.* 28 (5) (2009) 595–599, <http://dx.doi.org/10.1177/0278364909103911>.



aerodynamics.

Alex Fisher obtained his Bachelor's degree in Aerospace Engineering in 2008 from RMIT in 2008. Following a short stint as an embedded software engineer at BAE Systems, he returned to RMIT to complete a Ph.D. in the area of flapping wing aerodynamics. He was then employed as a Research Fellow at RMIT, conducting research and teaching in the area of unmanned aircraft systems (UAS). Alex is currently working on improving autonomous operation of small UAS in urban environments. His areas of interest include trusted autonomy, embedded systems and avionics for small UAS, and



collaborates with a wide variety of national and international experts in autonomous-system technologies.

Dr Jennifer Palmer holds a Bachelor of Science in Nuclear Engineering and a Master of Science in Aerospace and Mechanical Engineering, as well as a Doctorate in Mechanical Engineering from Stanford University. Over the last decade Dr Palmer has worked in the area of autonomous aircraft systems at the Defence Science and Technology Group in Melbourne, Australia, where her work has focussed on, amongst other things, hybrid power for small unmanned aircraft and flapping flight. Her latest research has been on the development of autonomous systems for urban operations. Dr Palmer



for data-ferrying and the exploration of unknown environments.

In 2021, Ricardo joined the Cognitive Robotics Group at the Oxford Robotics Institute as a PhD student. His research project is on the use of robust causal inference methods to assure the autonomy of mobile robots conducting inspection tasks.

Ricardo Cannizzaro is a Defence research scientist at the Australian Defence Science and Technology Group, where his research focusses on the use of teams of autonomous aerial and ground robots to conduct operations in cluttered, complex, and highly dynamic environments, such as the urban terrain. His research has covered a range of robotics problems, including: multi-modal sensing and navigation in mixed GNSS-available environments, path- and trajectory-planning methods for aerial robot navigation, decentralised multi-robot planning, and robotic swarming methods



Madeleine Cochrane is a researcher with the Defence Science and Technology Group in Australia. She received her honours degree in robotics and mechatronics engineering and bachelor's degree in computer science from Swinburne University of Technology in 2018. Her current research focuses on multi-agent decision making and shared contextual awareness.